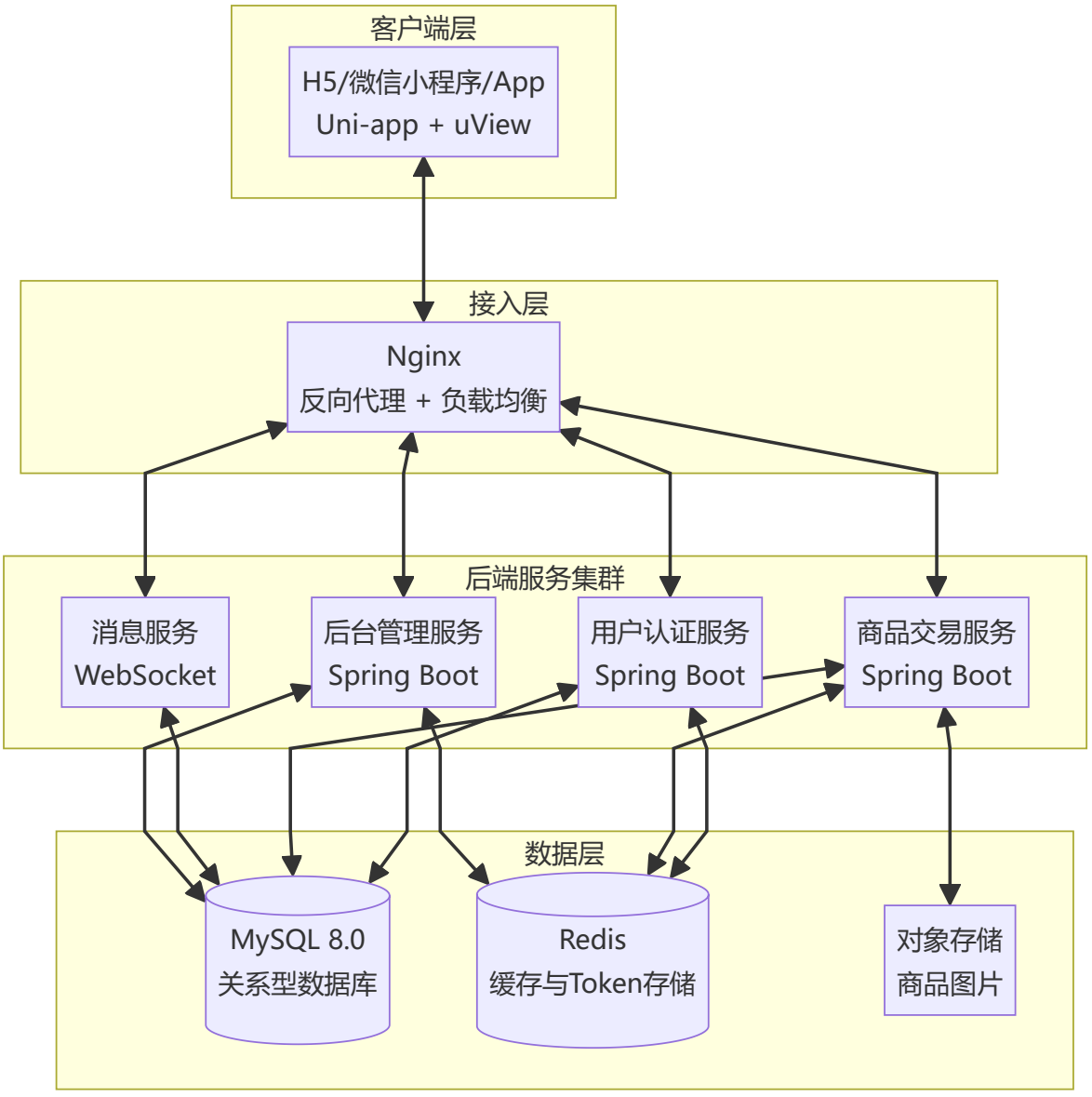
基于Uni-app的校园二手交易平台系统设计说明书

文档版本	修改日期	修改人	修改说明
V1.0	2026年3月28日	项目团队	初稿创建，明确系统架构、功能与数据库设计

1. 系统体系架构

本系统采用前后端分离的B/S架构，遵循高内聚、低耦合的设计原则。前端使用Uni-app实现跨平台（H5、微信小程序、App）的移动端展示与交互，后端基于Spring Boot构建RESTful API，为前端提供数据服务。

1.1 总体架构图



1.2 架构分层说明

- **客户端层**：基于Uni-app框架开发，一套代码编译为H5、微信小程序和App，覆盖多端用户。使用uView UI库保证界面美观和一致性。
- **接入层**：Nginx作为Web服务器和反向代理服务器，处理静态资源请求，并将动态请求转发至后端服务集群。
- **业务层**：采用Spring Boot构建微服务模块。
  - **用户认证服务**：处理微信授权登录、学号实名认证、JWT Token签发与校验。
  - **商品交易服务**：实现商品发布、检索、订单管理、收藏、评价等核心交易逻辑。
  - **消息服务**：基于WebSocket协议，维护与客户端的连接，实现买卖双方实时通信。
  - **后台管理服务**：为管理员提供用户管理、商品审核、数据统计等接口。
- **数据层**：
  - **MySQL**：存储用户、商品、订单等核心业务数据。
  - **Redis**：用于存储高频访问数据（如首页推荐商品、用户会话Token），缓解数据库压力，提升响应速度。
  - **对象存储**：统一存储用户上传的商品图片，保证图片访问速度和数据安全。

2. 系统功能结构（层次结构）

系统功能分为“用户端”和“管理端”两大模块，具体层次结构如下：

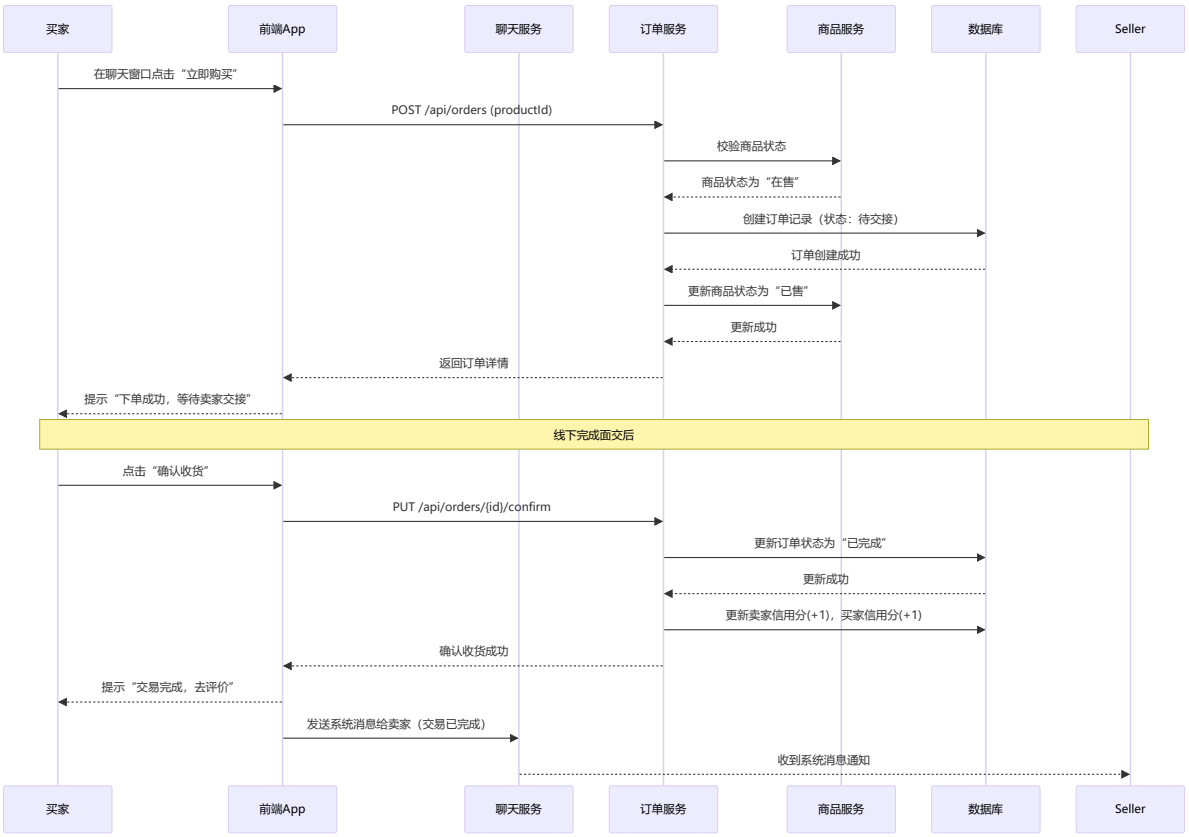
```
校园二手交易平台
├── 用户端
|   ├── 用户认证与中心
|   |   ├── 微信授权登录
|   |   ├── 学号实名认证
|   |   ├── 个人资料管理
|   |   ├── 我的收藏
|   |   ├── 我的发布
|   |   ├── 我的订单
|   |   └── 信用分查看
|   ├── 商品交易
|   |   ├── 商品发布（图片/描述/价格/分类/面交点）
|   |   ├── 商品编辑/下架/标记已售
|   |   ├── 商品浏览（首页推荐/分类/搜索）
|   |   ├── 商品详情
|   |   ├── 商品收藏与留言
|   |   └── 地图找商品（LBS）
|   └── 交易沟通
|       ├── 即时通讯（文本/图片/位置）
|       ├── 生成订单
|       ├── 订单状态流转（待交接 -> 已完成/已取消）
|       └── 互评与信用更新
└── 管理端
    ├── 用户治理
    |   ├── 用户列表查看
    |   ├── 用户状态管理（封禁/解封）
    |   └── 违规记录查询
    ├── 内容审核
    |   ├── 商品审核（通过/驳回）
```

```
    |   └── 商品分类管理（增/删/改/查）
    └── 运营监控
        ├── 数据统计看板
        |   ├── 新增用户趋势
        |   ├── 活跃度分析
        |   └── 交易量统计
        └── 系统日志审计
```

3. 系统用例的时序图以及说明

选取核心的“下单与确认收货”流程进行时序图描述。

3.1 下单与确认收货时序图



3.2 时序图说明

1. **下单阶段**：买家在聊天窗口发起购买请求，订单服务首先检查商品状态，确保商品未被他人购买。创建订单时，订单状态为“待交接”，并立即将商品状态更新为“已售”，实现库存锁定，防止超卖。
2. **确认收货阶段**：线下面对面交易完成后，由买家在订单列表中手动确认收货。该操作触发订单状态流转，并自动更新买卖双方的信用分，激励用户诚信交易。同时，系统会通过聊天服务向卖家推送交易完成的消息，形成闭环。

4. 复杂功能的算法设计

这里以“信用分计算与影响”和“首页轻量推荐”为例。

4.1 信用分计算模型

算法思路：信用分初始为100分。通过用户的交易行为和评价动态调整。旨在建立“良币驱逐劣币”的机制。

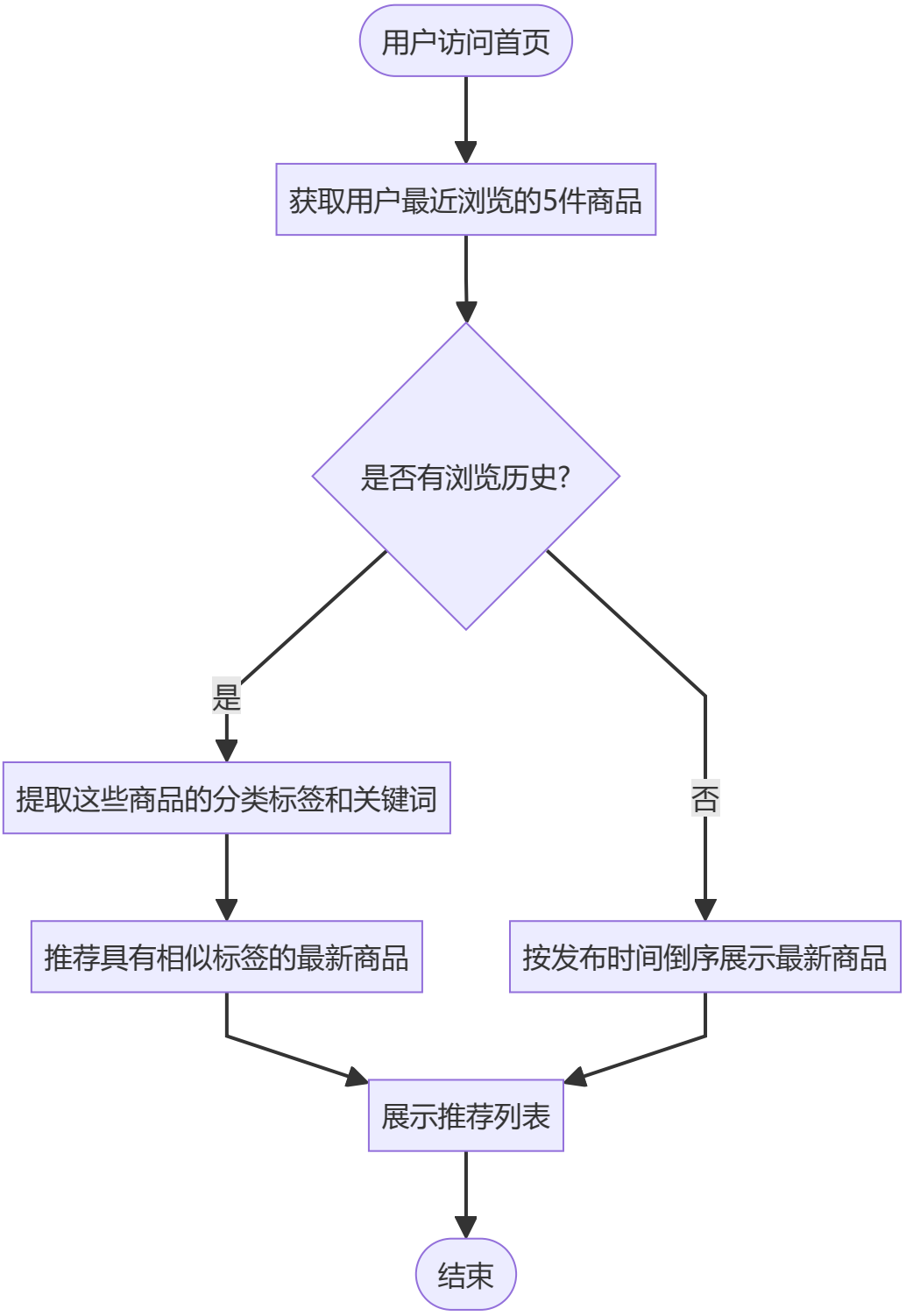
伪码表示：

```
FUNCTION updateCreditScore(userId, eventType):
    user = getUserById(userId)
    score = user.credit_score

    // 加分项
    IF eventType == "RECEIVE_GOOD_REVIEW":    // 作为卖家收到好评
        score = score + 5
    IF eventType == "GIVE_GOOD_REVIEW":      // 作为买家给出好评（体现履约）
        score = score + 2
    IF eventType == "COMPLETE_ORDER":        // 交易顺利完成
        score = score + 3

    // 减分项
    IF eventType == "RECEIVE_BAD_REVIEW":     // 收到差评
        score = score - 10
    IF eventType == "ORDER_CANCEL_BY_BUYER":  // 买家无故取消订单
        score = score - 2
    IF eventType == "ORDER_CANCEL_BY_SELLER": // 卖家无故取消订单
        score = score - 5
    IF eventType == "VIOLATION":              // 管理员判定违规
        score = score - 30

    // 边界控制
    score = max(0, min(score, 100))

    // 权限控制
    IF score < 60:
        setUserPublishPermission(userId, FALSE) // 禁止发布
    ELSE:
        setUserPublishPermission(userId, TRUE)

    saveUserCreditScore(userId, score)
    RETURN score
```

4.2 首页“猜你喜欢”推荐策略（轻量级）

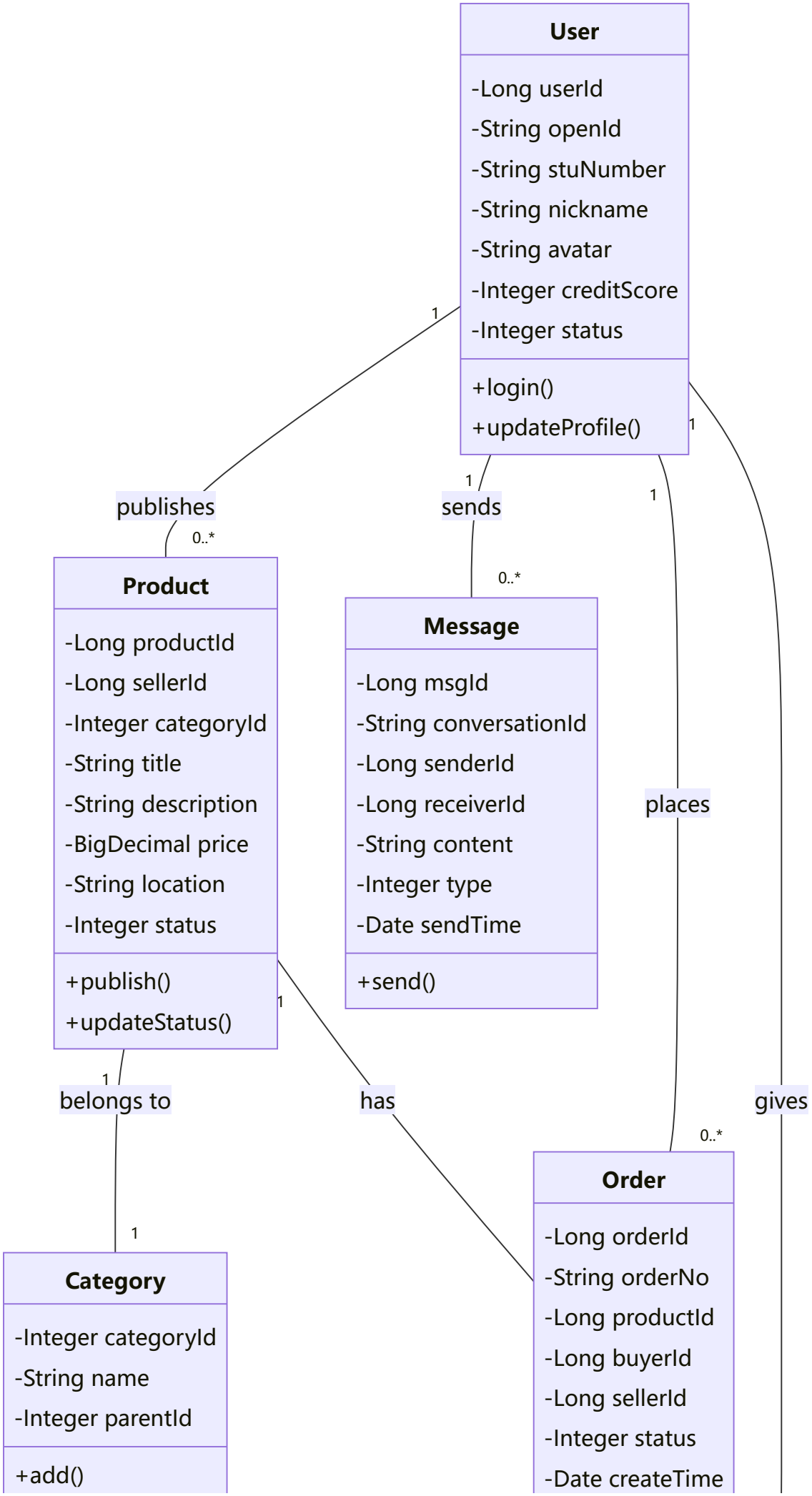
算法思路：基于内容的推荐。根据用户的最近浏览记录和收藏记录，匹配相似标签的商品。如果没有用户行为，则展示最新发布的商品。

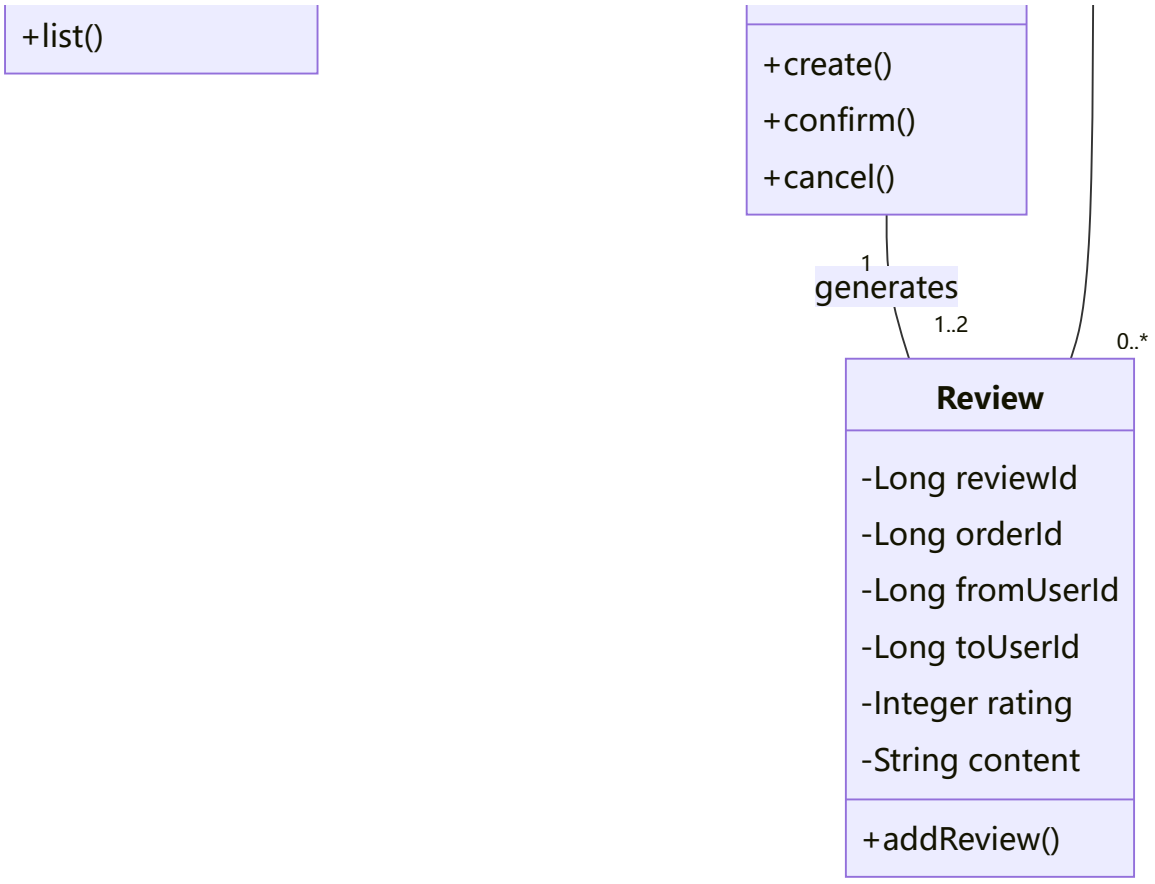
流程图：



5. 面向对象方法类图的详细设计

根据ER图和数据字典，设计核心实体类。


User
-Long userId
-String openId
-String stuNumber
-String nickname
-String avatar
-Integer creditScore
-Integer status
+login()
+updateProfile()
publishes
sends
places
gives
Product
-Long productId
-Long sellerId
-Integer categoryId
-String title
-String description
-BigDecimal price
-String location
-Integer status
+publish()
+updateStatus()
Message
-Long msgId
-String conversationId
-Long senderId
-Long receiverId
-String content
-Integer type
-Date sendTime
+send()
belongs to
has
Order
-Long orderId
-String orderNo
-Long productId
-Long buyerId
-Long sellerId
-Integer status
-Date createTime
Category
-Integer categoryId
-String name
-Integer parentId
+add()
1
0..*
1
0..*
1
1
1
0..*
1
1



6. 接口设计

遵循RESTful API设计规范，使用JSON格式进行数据交互。

6.1 核心API接口列表

模块	方法	URL	描述	请求参数示例	返回数据示例
认证	POST	/api/auth/wechat-login	微信登录	`{ "code": "wx_code" }`	`{ "token": "jwt_token", "userId": 1 }`
认证	POST	/api/auth/student-verify	学号认证	`{ "stuNumber": "2023302112018", "realName": "张三" }`	`{ "success": true }`
商品	POST	/api/products	发布商品	`{ "title": "二手教材", "price": 20, "categoryId": 1, ... }`	`{ "productId": 101 }`
商品	GET	/api/products	商品列表/搜索	`?keyword=教材&category=1&page=1&size=10`	`{ "list": [], "total": 100 }`
订单	POST	/api/orders	创建订单	`{ "productId": 101 }`	`{ "orderId": 10001, "status": 1 }`
订单	PUT	/api/orders/{id}/confirm	确认收货	`{ "id": 10001 }`	`{ "success": true }`
聊天	WS	/ws/chat	WebSocket连接	`?token=jwt_token`	建立连接，进行消息推送

模块	方法	URL	描述	请求参数示例	返回数据示例
管理	PUT	/api/admin/products/{id}	商品审核	`{ "status": 1, "remark": "审核通过" }`	`{ "success": true }`

6.2 错误码规范

遵循文档中的错误码定义，确保前端能正确处理异常。

错误码	HTTP状态码	场景	提示文案
40101	401	未登录或令牌失效	登录状态失效，请重新登录
40301	403	权限不足（如未认证学号）	请先完成学号认证
40021	400	参数校验失败	请检查输入内容后重试
40401	404	资源不存在	该商品或订单不存在
50001	500	系统异常	系统繁忙，请稍后再试

7. 数据库物理设计

基于ER图和需求文档中的“数据字典”，给出核心表的完整物理设计。

7.1 用户表 (user)

字段名	类型	约束	描述
user_id	BIGINT	PK, AUTO_INCREMENT	用户主键ID
open_id	VARCHAR(64)	UNIQUE	微信唯一标识
stu_number	VARCHAR(20)	UNIQUE, NOT NULL	学号
nickname	VARCHAR(50)	NOT NULL	昵称
avatar	VARCHAR(255)		头像URL
credit_score	INT	DEFAULT 100	信用分 (0-100)
status	TINYINT	DEFAULT 1	状态：1-正常，0-封禁
create_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	注册时间

7.2 商品表 (product)

字段名	类型	约束	描述
product_id	BIGINT	PK, AUTO_INCREMENT	商品ID
seller_id	BIGINT	FK, NOT NULL	卖家ID
category_id	INT	FK, NOT NULL	分类ID

字段名	类型	约束	描述
title	VARCHAR(100)	NOT NULL	标题
description	TEXT		商品描述
price	DECIMAL(10,2)	NOT NULL	售价
location	VARCHAR(100)		面交地点
image_urls	JSON		图片URL列表，如 `["url1","url2"]`
status	TINYINT	DEFAULT 0	状态：0-待审核，1-在售，2-已售，3-下架，4-驳回
create_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	发布时间

7.3 订单表 (order)

字段名	类型	约束	描述
order_id	BIGINT	PK, AUTO_INCREMENT	订单ID
order_no	VARCHAR(64)	UNIQUE, NOT NULL	唯一订单号
product_id	BIGINT	FK, NOT NULL	商品ID
buyer_id	BIGINT	FK, NOT NULL	买家ID
seller_id	BIGINT	FK, NOT NULL	卖家ID
status	TINYINT	DEFAULT 1	状态：1-待交接，2-已完成，3-已取消
remark	VARCHAR(255)		订单备注
create_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	下单时间
finish_time	DATETIME		完成/取消时间

7.4 聊天消息表 (message)

字段名	类型	约束	描述
msg_id	BIGINT	PK, AUTO_INCREMENT	消息ID
conversation_id	VARCHAR(64)	NOT NULL	会话ID（由双方ID组成）
sender_id	BIGINT	FK, NOT NULL	发送者ID
receiver_id	BIGINT	FK, NOT NULL	接收者ID

字段名	类型	约束	描述
content	TEXT	NOT NULL	消息内容
msg_type	TINYINT		类型：1-文本，2-图片，3-位置
is_read	TINYINT	DEFAULT 0	是否已读
send_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	发送时间

8. UI（界面）设计

界面设计遵循移动端设计规范，追求简洁、清晰、易用。

8.1 主要界面设计

1. **首页**：
    - **顶部**：搜索框（支持关键词模糊搜索）。
    - **中部**：分类导航栏（教材、数码、生活、其他等），采用横向滑动布局。
    - **主体**：推荐商品瀑布流。每个商品卡片包含：商品图片、标题、价格、卖家信用分徽章。
    - **底部**：TabBar导航栏（首页、消息、发布、订单、我的）。
2. **商品发布页**：
    - **图片上传区**：支持从相册选择或拍照，最多9张，可拖动排序。
    - **表单**：标题、描述、价格、分类选择器、面交地点输入框。
    - **发布按钮**：位于页面底部，点击后提交审核。
3. **聊天页面**：
    - **界面布局**：类似微信聊天，顶部显示对方昵称和信用分。
    - **消息气泡**：区分自己的消息（绿色）和对方的消息（白色）。
    - **输入区**：支持文本输入、图片发送（调用相册/相机）、位置发送（调用地图选择校内点）。
    - **交易卡片**：当用户在聊天中点击“立即购买”或发送购买链接时，聊天界面会生成一个商品卡片，点击可跳转或确认订单。
4. **管理端（Web）**：
    - **登录页**：管理员专用登录页面。
    - **仪表盘**：数据看板，以折线图展示用户增长趋势，以饼图展示商品分类占比，以表格展示待审核商品列表。
    - **商品审核页**：展示待审核商品，管理员可放大查看图片，点击“通过”或“驳回”，并填写驳回理由。